

micsPlusTableR

User's Guide - Version 0.2.7

MICS Team

2026-08-27

Table of contents

1	What the system does	2
2	Installation and startup	2
2.1	Install from GitHub	2
2.2	Install from a local folder	3
2.3	Start the application	3
2.4	Update the package	4
2.5	Confirm the installed functions	4
3	Use the package without the application	4
4	Prepare the inputs	6
4.1	Tabulation plan workbook	6
4.2	Statistic types	7
4.3	Totals, groups, indents, and suppression	7
4.4	Survey data files	8
4.5	Preparation folder contract	8
4.6	Extra tables and FIES	9
5	Run the application	10
5.1	Data Preparation	10
5.2	Tabulator	10
5.3	Consistency Checks	10
5.4	Write to Excel	10
5.5	Multi-Sheet Tabulator	11
5.6	Data Exploration, Data View, and Long Format Data	11
6	Output workbooks	11
7	Troubleshooting	12
7.1	The app does not start	12
7.2	Newly installed functions do not appear	12
7.3	The preparation script fails	12
7.4	A table cannot find a variable	12

7.5	A weight is missing or invalid	12
7.6	A consistency tab is red	12
7.7	The Excel preview is blank	13
7.8	The output workbook is locked	13
8	Statistical and security notes	13
8.1	Single-context engine: <code>tabulate_v2()</code>	14
8.2	Block-aware engine: <code>tabulate_h2()</code> and <code>calc_cells()</code>	14
9	Production checklist	14

This guide explains how to install and use `micsPlusTableR`, prepare its three inputs, create and review MICS Plus tabulations, and diagnose common problems. It is written for analysts who maintain survey-specific preparation scripts as well as users who run approved inputs.

Important: A preparation script is executable R code. Use only scripts from a trusted source and review them before uploading.

1 What the system does

`micsPlusTableR` is an R package containing a Shiny application and the tabulation engine. The application coordinates three user-supplied inputs:

1. **Tabulation plan:** an Excel workbook that defines table structure, conditions, statistics, labels, formatting, and the table index.
2. **Survey preparation inputs:** household and household-member SPSS files, plus a selected preparation folder whose main R script creates the prepared `hh` and `hl` data.
3. **Additional table code:** optional code in the preparation workflow for tables that cannot be generated by the generic Excel-plan logic, including FIES processing where applicable.

The normal flow is:

```
Install package -> Start app -> Upload inputs -> Validate survey information ->
Tabulate -> Check -> Export
```

The package no longer installs dependencies while the app is starting and no longer requires users to source folders of functions. Package installation handles the declared dependencies, and each running Shiny session uses its own tabulation state.

Version 0.2.4 also removes the former project-level `child` folder. Its three roles are now part of the package: `DESCRIPTION` manages required packages, `mics_session()` loads an isolated tabulation engine, and `create_excel_workbooks()` creates working output files. Do not source `child/library.R`, `child/source-functions.R`, or `child/create-excel-files.R`; these files are not required after installation.

2 Installation and startup

2.1 Install from GitHub

The repository must first be published on GitHub. Replace `OWNER` with the GitHub organization or user that owns the repository.

```
install.packages("remotes")
remotes::install_github("OWNER/micsPlusTableR")
```

2.2 Install from a local folder

If the package source folder is already on the computer, install directly from that folder. The folder must contain DESCRIPTION, NAMESPACE, and the R directory.

```
install.packages("remotes")
remotes::install_local("C:/MICSPlus/micsPlusTableRv2")
```

On macOS or Linux, use the appropriate path, for example:

```
remotes::install_local("/Users/name/projects/micsPlusTableRv2")
```

Alternatively, install a built package archive without `remotes`:

```
install.packages(
  "micsPlusTableR_0.2.7.tar.gz",
  repos = NULL,
  type = "source"
)
```

The installation command is needed after the source code changes. Editing the source folder does not automatically update an already installed copy.

2.3 Start the application

Start the application with:

```
micsPlusTableR::run_app()
```

When `run_app()` starts, it uses `here::here()` to find the user's project root and automatically creates `micsPlusTableR-output` there. This works across Windows, macOS, and Linux. Generated Output and Formatted workbooks are written there. The folder is a runtime destination rather than part of the package source; if an empty copy is removed, the application creates it again at the next startup.

To choose a different default output folder:

```
micsPlusTableR::run_app(
  output_dir = "C:/MICSPlus/outputs"
)
```

On macOS or Linux, use the corresponding path format. The application creates the directory if it does not exist.

2.4 Update the package

Run the same installation command when a new version is released. For a reproducible production cycle, record the package version used for the final tables:

```
packageVersion("micsPlusTableR")
```

2.5 Confirm the installed functions

Restart R after installing or updating the package. An R session that loaded an older release keeps that release's namespace in memory, even if newer files have already been installed on disk. In RStudio, select **Session > Restart R** or press **Ctrl+Shift+F10** on Windows and Linux or **Cmd+Shift+F10** on macOS.

Verify the installed release and its public functions:

```
packageVersion("micsPlusTableR")
sort(getNamespaceExports("micsPlusTableR"))
```

The exported functions include `run_app()`, `mics_session()`, `prepare_mics_data()`, `tabulate_mics_table()`, `pivot_mics_table()`, `check_mics_table()`, and the Excel writing and comparison helpers documented in the next section.

If `remotes` reports that it is skipping a local installation while the source folder has changed, force one clean installation and restart R again:

```
remotes::install_local(
  "/path/to/micsPlusTableRv2",
  force = TRUE,
  upgrade = "never"
)
```

3 Use the package without the application

The same supported tabulation workflow is available from R scripts. This is useful for reproducible production runs, batch processing, diagnostics, and analyses that need the prepared data or tabulated cells outside Shiny.

Create one explicit session object for each survey run. The session holds the prepared data and engine state without adding objects to the global workspace.

```
library(micsPlusTableR)

job <- mics_session()

prepare_mics_data(
  job,
  hh_path = "inputs/hh.sav",
```

```

hl_path = "inputs/hl.sav",
prep_script = "inputs/survey-preparation/country_wave_prep.R",
output_dir = "micsPlusTableR-output"
)

result <- tabulate_mics_table(
  job, "inputs/tabulation-plan.xlsx", "1.1a"
)

preview <- pivot_mics_table(job, result, type = "header")
checks <- check_mics_table(job, result)

write_mics_table(
  job,
  destination = "outputs/Output Tables.xlsx",
  sheet = "1.1a",
  table = result
)

```

The main functions for work outside the application are:

Function	Purpose
<code>mics_session()</code>	Create isolated state for one survey workflow
<code>prepare_mics_data()</code>	Read survey files through the trusted prep script
<code>read_mics_tabulation()</code>	Read one table definition from the Excel plan
<code>tabulate_mics_table()</code>	Tabulate one sheet and retain its result
<code>pivot_mics_table()</code>	Create index, header, or logic-oriented previews
<code>check_mics_table()</code>	Run the applicable count and percentage checks
<code>write_mics_table()</code>	Write numeric or formatted cells to a workbook
<code>write_mics_footnotes()</code>	Add standard footnotes to a workbook sheet
<code>read_previous_mics_table()</code>	Read a prior table for comparison
<code>compare_mics_tables()</code>	Compare two table data frames cell by cell
<code>mics_table_name()</code>	Return the current table identifier
<code>create_excel_workbooks()</code>	Create collision-safe workbook copies

`write_mics_table()` writes to an existing workbook, so create a working copy of the plan first. The helper below creates timestamped copies safely:

```

books <- create_excel_workbooks(
  path_tab_excel = "inputs/tabulation-plan.xlsx",
  output_dir = "outputs",
  country_code = "MNG",
  period = "2025-26",
  wave = "Wave 1"
)

```

```
write_mics_table(
  job,
  destination = books$output,
  sheet = "1.1a",
  table = result
)
```

Do not run two tabulations at the same time with the same session object. Use a separate `mics_session()` for each concurrent survey or workflow. The lower level engine functions remain internal because they depend on mutable engine state; the public wrappers above are the supported scripting interface.

4 Prepare the inputs

4.1 Tabulation plan workbook

The workbook must contain an **IDX** sheet and one sheet for each table.

4.1.1 Survey information on IDX

The application checks these cells:

Cell	Required content	Example
B2	Country	Mongolia
B3	Survey name	MICS Plus
B4	Period	2025-26
B5	Wave	Wave 2

The table list on **IDX** must use names beginning with **Table**, followed by the table code and table title. Every table sheet that should be processed must be listed consistently.

4.1.2 Minimum table-sheet structure

- Row 1 contains the table code and cell statistic types.
- Row 2 contains the table title.
- Row 3 starts the highest header level.
- One row between rows 4 and 7 contains the data/filter definition. It is identified by content such as `.sav`, `filter(...)`, or `mutate(...)`.
- The first table-wide filter is in column B of the condition row.
- A block-specific filter is placed at the beginning of its column block.
- Dataset and weight definitions are written in the same condition cell.
- Row and column conditions must be valid R expressions for the prepared data.

Do not insert manual line breaks in cells unless the plan convention explicitly requires them. Leave the active cell at **A1** before saving the workbook to make review easier.

4.1.3 Conditions and block behavior

The condition row is both executable logic and a compact description of each table block. A condition cell can identify the dataset, a `filter(...)` or `mutate(...)` expression, a `weight` by variable, and the final row or column condition. The first condition in column B establishes the table-wide context; conditions at the beginning of later column blocks refine that context.

The parser recognizes these established aliases:

- `total1 == 1` through `total5 == 1` are normalized to `total == 1`.
- `totalHL == 1` and `totalHL1 == 1` through `totalHL4 == 1` select the household-member total used by the horizontal engine.
- `ph` is reserved for extra-table placeholders and must not be used as an ordinary analytic condition.

Keep separate percentage distributions in separate blocks and end each block with its own 100 or 100.0 cell. When a block uses a different dataset, filter, or weight, declare that information at the beginning of the block rather than relying on a definition from a neighboring block.

4.2 Statistic types

The following types are recognized by the established engines. Availability can depend on the table direction and engine.

- `n`, `n2`: weighted count; `n2` is excluded from selected checks.
- `n_unw`, `n_unw2`: unweighted count; `n_unw2` is excluded from selected checks.
- `p`, `p(100)`: weighted percentage.
- `p_unw`, `p_unw(100)`: unweighted percentage.
- `mean`, `mean(var)`: mean or weighted mean of `var`.
- `mean_unw`, `mean_unw(var)`: unweighted mean.
- `median(var)`, `median_unw(var)`: weighted-context or unweighted median.
- `p_sum(var)`: weighted sum of `var` divided by the weight total, times 100.
- `100`, `100.0`: fixed total cell.
- `ph`: placeholder used by extra-table workflows.

Use the exact spelling and parentheses shown in the approved plans. Before introducing a new statistic, add a regression fixture and confirm how both the numerator and denominator should be defined.

4.3 Totals, groups, indents, and suppression

- A total condition is normally written as `total == 1`.
- `all` means no additional variable filter; it is not automatically a total.
- Separate a total row from the rows above it with an empty row when it defines a row-group consistency block.
- Percentage blocks expected to sum to 100 must end with their 100 cell.
- Use types such as `n2` or `p2` only where a cell must be excluded from a consistency check while retaining its tabulation behavior.
- Supported indentation levels for automatic checks are 1 and 2.
- Suppression requires an `n_unw` column before the `IDX` marker column.

4.4 Survey data files

Provide two SPSS `.sav` files:

- `hh`: household-level data.
- `hl`: household-member-level data.

Variable names used in the plan must exist after the preparation script runs. Weight variables must be numeric after preparation. Preserve value and variable labels when they are needed for Data Exploration or Data View.

4.5 Preparation folder contract

Select the preparation **folder** with the same native browser control used by the other app inputs. The folder is uploaded into the app session so the workflow behaves consistently on Windows, macOS, and Linux without fixed local paths or the nonstandard in-app filesystem browser.

The app chooses the main preparation script from top-level `.R` files:

- if there is only one top-level `.R` file, it is the main script;
- if there are several, exactly one filename must contain `prep` (case-insensitive); and
- if the choice is still ambiguous, the app reports the candidate filenames and asks you to keep one main prep-like file at the top level.

Use this structure:

```
survey-preparation/
|-- country_wave_prep.R      # detected as the main top-level prep script
`-- FIES-inputs/             # optional subfolder; use this exact folder name
    |-- fies-entry.R
    |-- fies-functions.R
    `-- fies-reference.RData
```

The application stages the uploaded folder for the current session, treats its root as `prep_dir`, and supplies `hh_path`, `hl_path`, `prep_dir`, and `prep_script` to the main script. The script must create `hh` and `hl` in its own execution environment.

```
hh <- haven::read_sav(hh_path) |>
  dplyr::mutate(total = 1)

hl <- haven::read_sav(hl_path) |>
  dplyr::mutate(total = 1)
```

Recommended rules:

- Use `package::function()` notation in production scripts.
- Do not call `install.packages()` from a preparation script.
- Do not use fixed paths for the uploaded `hh` and `hl` files.
- Keep helper scripts and reference assets below `FIES-inputs`, not beside the main prep script.
- Keep table-specific recodes documented near the extra table that uses them.
- Stop with a clear message when required variables are missing.

- Avoid writing files outside a user-selected output folder.

The app isolates ordinary objects created by the script within the current session. A script can still perform external actions such as reading other files or loading packages, which is why scripts must be trusted and reviewed.

4.6 Extra tables and FIES

For a table produced outside the generic logic, the preparation script can create:

```
tables_extra <- c("4.1")
extra_tables_dict <- list("4.1" = "tb_41")
tb_41 <- data.frame(...)
```

`tables_extra` lists the relevant sheet names. `extra_tables_dict` maps each sheet to the object holding the precomputed table. The object must match the row and column structure expected by the sheet.

FIES is a specialized extra-table workflow. When the selected preparation folder contains a top-level subfolder named `FIES-inputs`, the app:

1. exposes its full path as both `fies_inputs_dir` and `path_fies`;
2. exposes the staged preparation folder as `prep_dir`;
3. creates `FIES-outputs` under the configured main output folder only if it does not already exist; and
4. exposes that destination as both `fies_output_dir` and `path_fies_output`.

With the default app configuration, the output structure is:

```
micsPlusTableR-output/
|-- Output ... .xlsx
|-- Formatted ... .xlsx
`-- FIES-outputs/           # created only when FIES-inputs exists
    |-- *.csv
    |-- *.pdf
    |-- *.png
    `-- *.RData
```

The main prep script controls which FIES entry script runs. It can source a helper by name because the app searches the prep folder and then `FIES-inputs`:

```
# path_fies and path_fies_output are supplied by micsPlusTableR.
source("fies-functions.R")
source(file.path(path_fies, "fies-entry.R"))
```

Do not redefine `path_fies` or `path_fies_output` in the prep code. FIES code should write explicit artifacts with `file.path(path_fies_output, filename)`. The app also runs preparation with `FIES-outputs` as the temporary working directory, so legacy helper code that writes a relative filename is routed to the same place. Keep FAO-derived code and reference assets versioned with the survey preparation project, confirm package dependencies before running, and do not assume files from one country or wave are reusable without review.

5 Run the application

5.1 Data Preparation

1. Select the supported country/period and wave.
2. Upload the household `.sav` file.
3. Upload the household-member `.sav` file.
4. Upload the tabulation-plan workbook.
5. Select the preparation folder. Confirm that **Main prep** shows the intended top-level `.R` file and whether **FIES helpers** reports **FIES-inputs** or **none**.
6. Select **Set**.

The app loads the prepared data and compares the **IDX** metadata with the chosen survey settings. Resolve every mismatch before producing tables. Use the workbook preview to confirm that the expected sheets are available.

5.2 Tabulator

Select one sheet and select **Run**. Choose the preview mode:

- **Unformatted:** numeric output intended for checking.
- **Suppressed:** applies the established small-denominator display rules.
- **Formatted:** presentation-oriented preview.

The pivot mode controls whether the preview emphasizes worksheet indices, labels, or row/column logic. Review the table title and several cells against the source plan before continuing.

5.3 Consistency Checks

Select **Run Checks** after tabulating a sheet. The app checks applicable row, column, and indent groups for counts and percentages.

Tab color	Meaning
Green	The check ran and no difference exceeded the tolerance
Red	One or more differences exceeded the tolerance
No color	The check was not applicable or produced no check table

Select **Show only inconsistent cells** to focus on failures. A passing check does not prove that a table is conceptually correct; also review conditions, weights, denominators, missing-value rules, and the table title.

5.4 Write to Excel

Choose whether to create an Output workbook, a Formatted workbook, or both. Select **Create Excel File(s)** once for a production run. The application copies the plan and adds a timestamp to the workbook name. If a name already exists, a numeric suffix is used rather than silently overwriting it.

After a single table is reviewed, use **Write to Output Table** or **Write to Formatted Table**. The table is written to its matching worksheet.

5.5 Multi-Sheet Tabulator

Select one or more sheets, choose the write target, and select **Run and write selected**. The results panel reports each sheet, elapsed time, message, and consistency status. Run sheets sequentially within a session and do not close the browser or R process before completion.

Start with a small representative set after any package, plan, or prep-script change. Run the full workbook only after those tables pass review.

5.6 Data Exploration, Data View, and Long Format Data

- **Data Exploration** creates frequency or two-variable cross tables from the prepared `hh` or `hl` data. A typed filter must be a valid R expression.
- **Data View** displays selected prepared variables and their labels.
- **Long Format Data** runs all table sheets and combines cell-level results with survey, wave, sheet, and table identifiers.

These tools support diagnosis and review. They do not replace the approved tabulation plan or formal consistency checks.

6 Output workbooks

The package creates two workbook types from the uploaded plan:

- **Output Tables:** preserves unformatted numeric values for validation and downstream review.
- **Formatted Tables:** uses formatted and suppressed display values, hides the condition column, reduces the condition-row height, and adds standard footnotes where applicable.

The footnote writer scans the formatted values and adds only the standard notes whose markers occur in the table:

- () Figures that are based on 25-49 unweighted cases
- (*) Figures that are based on fewer than 25 unweighted cases
- - denotes 0 unweighted cases in the denominator

Keep any survey-specific footnotes in the tabulation plan. Do not give the last manual footnote a bottom border; the writer places the final table border after the standard notes it adds.

The workbook remains the operational record of a run. Keep the following items together for reproducibility:

- input `.sav` files or their controlled archive identifiers;
- the exact tabulation plan;
- the exact preparation and extra-table code;
- package version;
- generated Output and Formatted workbooks;
- review notes and approved exceptions.

7 Troubleshooting

7.1 The app does not start

Reinstall the package and restart R. Then verify:

```
packageVersion("micsPlusTableR")  
micsPlusTableR::run_app(launch.browser = TRUE)
```

If GitHub installation reports a dependency error, record the full message. Survey-specific scripts can require additional packages beyond the generic application dependencies.

7.2 Newly installed functions do not appear

If autocomplete shows only `run_app()` and `create_excel_workbooks()`, the current R session still has an older namespace loaded. Restart R, confirm that `packageVersion("micsPlusTableR")` reports 0.2.7, and inspect `getNamespaceExports("micsPlusTableR")`. If necessary, reinstall from the local folder with `force = TRUE` as shown under **Confirm the installed functions**, then restart R once more.

7.3 The preparation script fails

Check that:

- the selected folder contains the intended main `.R` file at its top level;
- if multiple top-level `.R` files exist, exactly one filename contains `prep`;
- the script uses the supplied `hh_path` and `hl_path`;
- it creates both `hh` and `hl`;
- required packages are installed and functions are namespaced;
- every selected variable exists in the corresponding data;
- fixed project paths have been removed or replaced by supplied paths;
- optional FIES helpers are in a top-level subfolder named exactly `FIES-inputs`; and
- FIES code does not overwrite `path_fies` or `path_fies_output`.

7.4 A table cannot find a variable

Compare the variable named in the error with the prepared data in Data View. Check spelling, capitalization, the selected dataset, and whether the variable is created before the table runs.

7.5 A weight is missing or invalid

Confirm the weight named in the condition row exists after all mutations and is numeric. For block-aware tables, inspect every block because later blocks can select a different weight.

7.6 A consistency tab is red

Review the displayed differences, then check the total condition, empty-row group boundaries, excluded `n2` or `p2` cells, block filters, and percentage denominators. Do not increase the tolerance simply to hide

an unexplained difference.

7.7 The Excel preview is blank

Confirm the uploaded workbook is a readable `.xls` or `.xlsx` file, contains at least one worksheet, and is not password protected. The preview uses a browser component, while the tabulation engine reads the workbook on the R side; report which of those two actions failed.

If one worksheet repeatedly fails while other sheets in the same workbook work, copy that worksheet's used cells into a new worksheet, preserve the required sheet name and formatting, and test the copy. This can isolate workbook-level cell metadata corruption. Keep the original file for diagnosis rather than overwriting it immediately.

7.8 The output workbook is locked

Close the workbook in Excel and try again. Writing to a workbook open in another program can fail, especially on Windows.

8 Statistical and security notes

The package intentionally retains two complementary tabulation engines:

- `tabulate_v2()` handles single-context, column-first tables.
- `tabulate_h2()` builds block-specific filter, calculation, and weight contexts and delegates cell calculations to `calc_cells()`.

`tabulate_mics()` selects the engine from the table direction read from the plan: direction `v` uses `tabulate_v2()`; every other direction uses `tabulate_h2()`. Both engines return the same long-format cell structure, but they interpret the table through different execution models.

Characteristic	<code>tabulate_v2()</code>	<code>tabulate_h2()</code> with <code>calc_cells()</code>
Intended table	One coherent data, filter, and calculation context	Multiple filter, calculation, or weight contexts across column windows
Evaluation order	Column first, then row	Row first, then column predicate
Preparation	Applies one consolidated row-calculation chain once	Builds a shared calculation chain plus block-specific calculations
Filtering	One primary filter, followed by each column condition	A separate filter chain for each detected block
Weight	Uses the primary context's weight	Resolves and validates the latest applicable weight in each block
Main advantage	Direct flow and rich built-in statistic parsing	Block-aware context and robust calculation/weight handling

8.1 Single-context engine: `tabulate_v2()`

`tabulate_v2()` is appropriate when the complete table uses one base dataset, one primary filter, and one unified row-calculation layer. It reads the column and row conditions, filters the base data, applies all row-derived calculations once, and then computes cells in nested loops. The outer loop creates each column subset; the inner loop evaluates each row statistic inside that subset.

This column-first model is useful for questions such as “within this column subgroup, what is the distribution across rows?” Its built-in statistic parser supports weighted and unweighted counts and percentages, means, unweighted means, medians, constants, and computed weighted-mean expressions. It assumes that the plan's row indices and parsed row conditions remain aligned and that the table does not require block-specific filter windows.

8.2 Block-aware engine: `tabulate_h2()` and `calc_cells()`

`tabulate_h2()` is appropriate when different column ranges need different filters, calculations, or weights. It identifies filter starts in the plan, assigns column windows to blocks, combines shared and block-specific calculations, resolves the applicable weight after those calculations, and validates that the weight column exists. It then delegates each block's cell statistics to `calc_cells()` and joins the results.

`calc_cells()` evaluates rows first. For each row subset, it applies every column predicate and calculates the requested count, percentage, mean, median, sum-based percentage, or constant. Therefore a percentage in this engine is the proportion satisfying the column condition within the selected row subset. The engine also applies the established display rules for zero denominators and small unweighted sample sizes.

The engines are intentionally kept separate. Their numerator and denominator models, filter staging, weight resolution, and calculation ordering are not interchangeable. Version 0.2.4 packages and isolates the established behavior instead of forcing a universal implementation that could change approved results. Changes to either engine require regression testing with representative single-context and multi-block tables.

The internal workflow follows a fixed sequence. `read_tabulation()` parses the worksheet into its original grid, row and column logic, block filters, weights, headers, empty rows and columns, indentation, group information, and formatting flags. `tabulate_mics()` selects the applicable engine, `pivot_table()` creates review views, the six check functions validate applicable totals, and the Excel writers persist approved results and footnotes. `clear_previous()` resets table state while retaining prepared survey data in the explicit package session.

These engine functions remain internal because they share mutable table state. Analysts working outside Shiny should use the exported explicit-session wrappers, which provide the same sequence with input validation and a stable interface.

The plan and preparation script contain executable R expressions. Treat both as code: review changes, use version control, restrict write locations, and run only trusted inputs. The current application is designed for sequential work inside each session. Do not use one session to run overlapping table jobs.

9 Production checklist

Before release of final tables, confirm all items below.

- The package version is recorded.
- IDX country, period, and wave match the selected survey.
- The preparation script completed without warnings that affect results.
- Required variables and weights are present and plausible.
- Representative tables were compared with an approved reference.
- Applicable row, column, and indent checks pass or have documented exceptions.
- Extra tables and FIES outputs were reviewed separately.
- Output and Formatted workbooks were created once for the production run.
- Standard and survey-specific footnotes are present.
- Final files and code are stored together in the controlled archive.